GPUMODE Lecture Study Notes: Lecture 86: Getting Started with CuTe DSL

Core Problem the Lecture Addresses

This lecture introduces **CuTe DSL**, a Python-facing programming system built on top of the lower layers of CUTLASS 4. The central problem is how to let performance-oriented engineers write GPU kernels with enough abstraction to be productive, while retaining explicit control over NVIDIA hardware features such as Tensor Memory Accelerator, shared-memory swizzling, tensor cores, tensor memory, memory barriers, warp specialization, software pipelining, and tile-level layout transformations.

The motivating context is the Blackwell NVFP4 competition, where high-performance solutions are expected to use newer NVIDIA kernel programming infrastructure. The lecture frames CuTe DSL as a middle ground between two extremes:

- **High-level compilers and code generators** such as PyTorch-level graph compilers or other DSLs allow fast algorithmic iteration, but often hide hardware-level decisions.
- **PTX or low-level CUDA assembly-like programming** exposes maximum control, but requires the programmer to manage too many hardware details manually.
- **CuTe DSL** attempts to provide expressive abstractions for tensor layouts, tiling, memory movement, and tensor-core invocation, while still letting the user select hardware instructions, tile shapes, memory layouts, synchronization policies, and pipeline strategies.

The core tension can be summarized as:

$$\text{Developer Productivity} \quad \text{versus} \quad \text{Speed-of-Light Hardware Control.}$$

CuTe DSL tries to move this trade-off frontier by making low-level hardware programming available from Python while preserving the composability of CUTLASS and CuTe layout algebra.

The main mental model is that CuTe DSL is not merely a high-level matrix multiplication API. It is a Python-accessible way to assemble GPU kernels from explicit tensor layouts, copy atoms, MMA atoms, tensor descriptors, shared-memory layouts, and synchronization primitives.

Required Background

CUTLASS, CuTe, and CuTe DSL

CUTLASS is NVIDIA's template-based C++ library for high-performance linear algebra kernels. Historically, CUTLASS has provided highly optimized GEMM, convolution, and epilogue components through C++ templates. The difficulty is that heavy template metaprogramming can be hard to debug, hard to compile quickly, and intimidating for researchers who want to iterate rapidly.

CuTe is the lower-level algebraic layer inside CUTLASS that describes tensors, layouts, tiling, partitioning, and instruction-level data movement. CuTe DSL exposes many of these lower-level ideas through Python.

The rough stack is:

High-level CUTLASS APIs and tuned kernels
CuTe programming model
CuTe atom abstractions for copy and MMA
NVIDIA hardware instructions: TMA, tensor cores, shared memory, tensor memory

CuTe DSL currently focuses on the lower layers: tensor-layout programming, explicit memory movement, tensor-core invocation, and low-level kernel construction.

Why Python?

The lecture emphasizes that traditional CUTLASS kernels can take significant compile time because of C++ templates. Python integration reduces friction for prototyping and makes it easier to integrate custom kernels into PyTorch-oriented workflows.

The key argument is:

$$\text{Lower compile friction} + \text{Python integration} \Longrightarrow \text{faster kernel iteration.}$$

However, CuTe DSL is still low-level compared with ordinary Python tensor programming. The user must understand hardware layout, memory spaces, synchronization, and kernel launch structure.

GPU Execution Model

A CUDA-style GPU program executes a kernel over a **grid** of thread blocks. Each thread block runs on a streaming multiprocessor, or SM. Threads are grouped into warps, usually of size 32. Multiple warps in a block may cooperate through shared memory and synchronization primitives.

Important objects include:

- **Thread**: the smallest program instance.
- **Warp**: a group of 32 threads executing in SIMT fashion.
- **Thread block**: a group of threads sharing block-local shared memory.
- **Grid**: the full set of blocks launched by a kernel.
- **SM**: hardware unit that schedules warps, manages registers, shared memory, tensor cores, and memory pipelines.

GPU Memory Hierarchy

The relevant memory hierarchy for this lecture is:

Global Memory → L2 Cache → Shared Memory → Tensor Memory or Registers → Tensor Core or ALU Comp

Different CuTe DSL examples move data among the following spaces:

- **Global memory**: large off-chip memory.
- **Shared memory**: programmer-managed on-chip memory inside an SM.
- **Registers**: per-thread storage.
- **Tensor memory**: Blackwell-era local memory space used by certain tensor-core workflows.

Performance Metrics

The lecture repeatedly relies on the distinction between memory-bound and compute-bound kernels. A useful performance model is the roofline model:

$$T_{\text{kernel}} \geq \max\left(\frac{\text{FLOPs}}{\text{Peak FLOP/s}}, \frac{\text{Bytes Moved}}{\text{Peak Bandwidth}}\right).$$

The arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

A kernel with low arithmetic intensity is usually memory-bound. A kernel with high arithmetic intensity may become compute-bound if tensor cores or ALUs are efficiently utilized.

Main Concepts

CuTe DSL as a Low-Level Python Kernel DSL

CuTe DSL lets the programmer write Python code that describes a GPU kernel, but the code is not ordinary Python tensor code. It is a staged DSL with compile-time and runtime behavior. The programmer describes:

- host-side setup,
- kernel launch configuration,
- tensor layouts,
- memory movement atoms,
- shared-memory layouts,
- tensor-core atoms,
- register tensors,
- tensor SSA values,
- synchronization and pipelines,
- epilogue computations.

The DSL has two important decorators:

- **`@cute.jit`**: a host-side or inline DSL function decorator used to prepare parameters, compute launch dimensions, construct descriptors, and call kernels.
- **`@cute.kernel`**: the GPU kernel entry point, conceptually similar to a CUDA `__global__` function.

A simplified conceptual form is:

```
@cute.jit
def launch_kernel(A, B, C, problem_shape, static_config):
    grid = compute_grid(problem_shape, static_config)
    block = compute_block(static_config)
    kernel(A, B, C, problem_shape, static_config).launch(
        grid=grid,
        block=block
    )

@cute.kernel
```

```
def kernel(A, B, C, problem_shape, static_config):
    # Device-side tensor layout and computation logic.
    pass
```

The key division is that `@cute.jit` can be used for host-side staging and inline helper functions, while `@cute.kernel` represents the actual GPU entry point.

CuTe Layout Algebra

A major idea in CuTe is that tensors are not just arrays. They are combinations of memory iterators and layouts. A tensor describes how logical coordinates map to physical memory addresses.

A tensor can be modeled as:

$$\text{Tensor} = (\text{Iterator}, \text{Layout}).$$

The layout maps a coordinate tuple to an offset:

$$\text{offset} = \mathcal{L}(i_0, i_1, \ldots, i_{r-1}).$$

For a row-major matrix $A \in \mathbb{R}^{M \times K}$, the ordinary layout is:

$$\mathcal{L}_{\text{row-major}}(m, k) = mK + k.$$

For a column-major matrix, it is:

$$\mathcal{L}_{\text{column-major}}(m, k) = kM + m.$$

CuTe layout algebra generalizes this to hierarchical tiling, slicing, partitioning, swizzling, and instruction-level fragments.

Tiling and Partitioning

The first example in the lecture is a GEMV-like or simple GEMM-like kernel that uses global loads, register computation, and global stores. Even though it avoids advanced tensor cores, it teaches the fundamental CuTe pattern:

1. Construct a global tensor.
2. Tile the global tensor into block-sized regions.
3. Slice the tensor by block coordinates and thread coordinates.
4. Load the slice into registers or tensor SSA values.
5. Compute.

6. Store results back to global memory.

Suppose $A \in \mathbb{R}^{M \times K}$ is tiled by (B_M, B_K). The number of tiles is:

$$T_M = \left\lceil \frac{M}{B_M} \right\rceil, \qquad T_K = \left\lceil \frac{K}{B_K} \right\rceil.$$

A global coordinate (m, k) can be decomposed into tile coordinates and intra-tile coordinates:

$$m = t_m B_M + r_m, \qquad k = t_k B_K + r_k,$$

where

$$0 \le r_m < B_M, \qquad 0 \le r_k < B_K.$$

The tile coordinate (t_m, t_k) selects the block, while (r_m, r_k) selects the element inside the block.

In CuTe-style thinking, this coordinate decomposition is not merely arithmetic. It is encoded through tensor layout transformations and slicing operations.

Tensor SSA Values

The lecture introduces **tensor SSA** values as immutable value-semantic objects representing local tensor data. SSA stands for static single assignment. In practice, tensor SSA values provide a vectorized way to express loads, type conversions, and elementwise operations.

A typical flow is:

$$\text{Global Tensor} \to \text{Load} \to \text{Tensor SSA} \to \text{Elementwise Operations} \to \text{Store}.$$

For example, if x is loaded from global memory and converted from low precision to FP32, the conceptual operation is:

$$x_{\text{fp32}} = \text{convert}_{\text{fp32}}(x_{\text{lowp}}).$$

Then elementwise operations can be written at the tensor level:

$$z_i = f(x_i, y_i).$$

Instead of manually writing a scalar loop over each element, the DSL expresses this as tensor operations over a local fragment.

Control Flow

CuTe DSL supports familiar control flow constructs such as `for`, `while`, and `if`. The compiler distinguishes compile-time static conditions from runtime conditions.

If a condition is known at compile time, the DSL can evaluate or specialize the control flow during compilation. If the condition depends on runtime values, the compiler emits runtime IR control flow.

The conceptual distinction is:

$$\begin{aligned} \text{Compile-time condition} &\implies \text{Python-side specialization} \\ \text{Runtime condition} &\implies \text{Generated device control flow} \end{aligned}$$

CuTe DSL also provides `range`-like constructs with extra support for unrolling and pipelining.

Software Pipelining

The lecture explains a loop of the form:

```
for k_tile in cute.range(num_k_tiles):
    load_A_tile()
    load_B_tile()
    compute_tile()
```

This is simple but inefficient because computation waits for memory movement each iteration. Software pipelining overlaps the load of future tiles with the computation of the current tile.

A conceptual pipelined version is:

```
prefetch_tile(0)
prefetch_tile(1)

for k_tile in cute.range(num_k_tiles):
    wait_for_tile(k_tile)
    compute_tile(k_tile)
    prefetch_tile(k_tile + pipeline_depth)
```

If memory load time is T_{load} and compute time is T_{compute}, a non-overlapped loop costs approximately:

$$T_{\text{serial}} = N_K \left(T_{\text{load}} + T_{\text{compute}} \right).$$

A well-overlapped pipeline costs approximately:

$$T_{\text{pipeline}} \approx T_{\text{prologue}} + N_K \max(T_{\text{load}}, T_{\text{compute}}) + T_{\text{epilogue}}.$$

This is one of the most important optimization ideas in the lecture.

TMA: Tensor Memory Accelerator

TMA is a hardware feature that moves multidimensional tensor tiles from global memory to shared memory efficiently. Instead of each thread manually issuing scalar or vector loads, TMA uses a descriptor describing the tensor shape, strides, tile shape, and destination shared-memory layout.

The host side generally constructs:

$$\text{TMA Atom+Global Tensor+Shared Memory Layout+Tile Shape} \Longrightarrow \text{TMA Descriptor.}$$

On the device side, the kernel uses the descriptor to copy tiles:

$$\text{Global Memory Tile} \rightarrow \text{Shared Memory Tile.}$$

The lecture emphasizes that TMA setup has two sides:

- **Host side**: choose TMA atom, create descriptor, define shared-memory layout and swizzle.
- **Device side**: partition tensors, select tile coordinates, issue the copy operation.

Shared-Memory Swizzling

Shared memory is divided into banks. If multiple threads access different addresses that map to the same bank in the same transaction, bank conflicts occur. A swizzled layout changes how logical coordinates map to physical shared-memory addresses to reduce conflicts for the access pattern used by tensor cores or copy instructions.

A simplified bank index model is:

$$\text{bank}(a) = \left\lfloor \frac{a}{w} \right\rfloor \bmod N_{\text{banks}},$$

where a is the byte address, w is the bank word width, and N_{banks} is the number of banks. A swizzle transforms coordinates:

$$(m, k) \mapsto (m', k')$$

so that the physical bank distribution is more favorable.

The lecture's key point is that global memory may be row-major or column-major, but the optimal shared-memory layout for tensor-core consumption may

be different. TMA can move data from global memory into a swizzled shared-memory layout so that later tensor-core instructions can read it efficiently.

Copy Atoms

CuTe DSL uses **copy atoms** to represent low-level data movement instructions. The pattern appears repeatedly:

1. Pick the correct copy atom.
2. Build a tiled copy object.
3. Partition the source tensor.
4. Partition the destination tensor.
5. Call `cute.copy`.

The abstract pattern is:

$$\text{Atom} \to \text{Tiled Copy} \to (\text{Partitioned Source}, \text{Partitioned Destination}) \to \text{Copy}.$$

This applies to several memory paths:

$$\begin{aligned} \text{Global} &\to \text{Shared} \\ \text{Shared} &\to \text{Tensor Memory} \\ \text{Tensor Memory} &\to \text{Registers} \\ \text{Registers} &\to \text{Global} \end{aligned}$$

MMA Atoms and Tensor Cores

For GEMM-like computation, CuTe DSL exposes tensor-core operations through MMA atoms. The programmer chooses a tensor-core instruction shape based on the problem dimensions, data types, and desired tile shape.

A GEMM computes:

$$C_{M\times N} = A_{M\times K} B_{K\times N}.$$

A tiled tensor-core operation computes a small fragment:

$$C_{\text{tile}} \leftarrow C_{\text{tile}} + A_{\text{frag}} B_{\text{frag}}.$$

The total computation is:

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

For a tiled kernel:

$$C_{m,n} = \sum_{t_k=0}^{T_K-1} \sum_{r_k=0}^{B_K-1} A_{m,t_k B_K+r_k} B_{t_k B_K+r_k,n}.$$

The tensor-core atom must match hardware-supported instruction shapes. The lecture stresses that different tensor-core operations may be better or worse depending on the problem. For example, if $N = 1$, using a wide tensor-core shape in the N-dimension may waste compute.

Scaling Factors and Tensor Memory

For NVFP4-style computation, scaling factors for A and B may need to be loaded through shared memory and then moved into tensor memory before the tensor-core instruction can consume them.

The relevant path is:

$$\text{Global Memory} \rightarrow \text{Shared Memory} \rightarrow \text{Tensor Memory} \rightarrow \text{Tensor Core}.$$

If x is stored in a low-precision format with scale s, a conceptual dequantization is:

$$x_{\text{real}} = s \cdot x_{\text{quant}}.$$

For block-scaled formats, the scale may be shared across a group of values:

$$x_i \approx s_g q_i, \qquad i \in \mathcal{G}_g,$$

where $\mathcal{G}_g$ is the group controlled by scale s_g.

Hardware-Level Explanation

What Happens Inside the GPU

A high-performance CuTe DSL GEMM-like kernel typically proceeds as follows:

1. Each CTA, or cooperative thread array, is assigned an output tile.
2. The CTA computes its tile coordinate from the block index.
3. TMA descriptors describe how to load A, B, and scale tensors from global memory.
4. Producer warps issue TMA loads into shared memory.

5. Shared-memory layouts are swizzled to avoid bank conflicts during tensor-core consumption.
6. Some operands, such as scale tensors, are copied from shared memory to tensor memory.
7. Consumer warps issue tensor-core MMA instructions.
8. Accumulators reside in tensor memory or registers depending on the operation.
9. The epilogue moves data from tensor memory to registers, applies optional elementwise operations, and stores to global memory.

SM-Level Concurrency

The SM can overlap different types of work:

$$\text{TMA copy} \parallel \text{Tensor-core MMA} \parallel \text{Epilogue copy}$$

if the kernel is structured correctly. This motivates both software pipelining and warp specialization.

Warp Specialization

Warp specialization assigns different warps different roles. For example:

$$\begin{aligned} \text{Producer warp group} &: \text{issue TMA loads} \\ \text{Consumer warp group} &: \text{issue tensor-core MMA} \\ \text{Epilogue warp group} &: \text{move accumulators and store results} \end{aligned}$$

The value is that asynchronous instructions can proceed concurrently. TMA and tensor-core operations do not require every warp to execute identical work. A warp-specialized kernel can better hide memory latency and keep tensor cores fed.

Memory Barriers

Pipelined and warp-specialized kernels require synchronization to preserve correctness. If a consumer warp reads shared memory before a producer warp has completed a TMA load, the result is incorrect. Therefore, memory barriers coordinate producer and consumer phases.

The logical ordering is:

$$\text{Producer writes tile} \rightarrow \text{Barrier arrival} \rightarrow \text{Consumer waits} \rightarrow \text{Consumer reads tile.}$$

For S pipeline stages, the kernel often allocates a set of barriers:

$$\{\beta_0, \beta_1, \ldots, \beta_{S-1}\}.$$

At K-tile iteration t, the active stage is:

$$s(t) = t \bmod S.$$

This allows circular reuse of shared-memory buffers.

Shared-Memory Stage Count

If each A-tile needs B_A bytes and each B-tile needs B_B bytes, then S stages require approximately:

$$M_{\text{smem}} = S(B_A + B_B + B_{\text{scale}} + B_{\text{metadata}}) + M_{\text{barriers}}.$$

Increasing S improves latency hiding but consumes more shared memory, which may reduce occupancy:

$$\text{Resident CTAs per SM} \leq \left\lfloor \frac{M_{\text{SM, shared}}}{M_{\text{smem per CTA}}} \right\rfloor.$$

Thus, pipeline depth is a performance trade-off.

Register Pressure

The lecture discusses epilogue strategies where accumulators are copied from tensor memory to registers. If too much of the output tile is loaded into registers at once, the kernel may experience register spilling.

If each thread holds R_{acc} accumulator registers, R_{tmp} temporary registers, and R_{base} base registers, then:

$$R_{\text{thread}} = R_{\text{base}} + R_{\text{acc}} + R_{\text{tmp}}.$$

Occupancy may be bounded by registers:

$$\text{Resident warps} \leq \left\lfloor \frac{R_{\text{SM,total}}}{32 R_{\text{thread}}} \right\rfloor.$$

A subtiled epilogue reduces R_{acc} by processing smaller output slices.

Mathematical Reasoning

GEMM Work and Data Movement

For $C = AB$, where $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, and $C \in \mathbb{R}^{M \times N}$, the FLOP count is approximately:

$$\text{FLOPs} = 2MKN.$$

If each matrix element occupies b bytes and each matrix is read or written once, a lower bound on memory traffic is:

$$\text{Bytes} \geq b(MK + KN + MN).$$

The arithmetic intensity is:

$$I_{\text{GEMM}} = \frac{2MKN}{b(MK + KN + MN)}.$$

For large square GEMM with $M = N = K = L$:

$$I_{\text{GEMM}} = \frac{2L^3}{3bL^2} = \frac{2L}{3b}.$$

Thus, large GEMMs become compute-intensive, but small or skinny GEMMs may remain memory-sensitive.

Tiled GEMM Reuse

A CTA computing a tile $C_{B_M \times B_N}$ over a K-tile of size B_K reads:

$$B_M B_K \quad \text{elements of } A$$

and

$$B_K B_N \quad \text{elements of } B.$$

It performs:

$$2B_M B_N B_K$$

FLOPs. The tile-level arithmetic intensity is approximately:

$$I_{\text{tile}} = \frac{2B_M B_N B_K}{b(B_M B_K + B_K B_N)} = \frac{2B_M B_N}{b(B_M + B_N)}.$$

This shows why larger B_M and B_N improve reuse, but only if shared memory, registers, and tensor-core shapes can support them.

Tile Shape Selection

Tile shape selection must balance:

Tensor Core Utilization, Memory Reuse, Shared Memory Usage, Register Pressure, Occupancy.

A wide MMA shape may waste work if the problem dimension is small. If a tensor-core atom computes a fragment of shape $M_a \times N_a \times K_a$, but the real problem has effective $N < N_a$, then the wasted fraction in the N-dimension is roughly:

$$\text{Waste}_N = 1 - \frac{N}{N_a}, \qquad N < N_a.$$

This motivates choosing different atom shapes for GEMV-like or skinny-matrix cases.

Batched GEMV-Like Computation

The first example resembles a batched GEMV or simple GEMM where each thread may process a row or partial row. If $y = Ax$, then:

$$y_m = \sum_{k=0}^{K-1} A_{m,k} x_k.$$

For a batched version:

$$y_{j,m} = \sum_{k=0}^{K-1} A_{j,m,k} x_{j,k}.$$

A block-level tile might assign:

$$(j, t_m, t_k)$$

to a CTA, where j is the batch index, t_m is the row tile, and t_k is the reduction tile.

Dual GEMM

The dual GEMM example computes two GEMM results sharing some dimension. Conceptually:

$$D_0 = AB_0, \qquad D_1 = AB_1.$$

Then an epilogue applies a nonlinear operation to one result and combines them:

$$C = \phi(D_0) \odot D_1,$$

where $\odot$ is elementwise multiplication and ϕ may represent a nonlinear function such as a gating operation.

For a SiLU-like gating function:

$$\phi(x) = x \cdot \sigma(x), \qquad \sigma(x) = \frac{1}{1 + \exp(-x)}.$$

Then:

$$C_{m,n} = (D_{0,m,n}\sigma(D_{0,m,n}))\, D_{1,m,n}.$$

The important implementation point is that the epilogue must load accumulator fragments, apply elementwise tensor SSA operations, and store results efficiently.

Grouped GEMM

Grouped GEMM computes many independent GEMMs:

$$C^{(g)} = A^{(g)}B^{(g)}, \qquad g = 0, 1, \ldots, G-1,$$

where each group may have different shapes:

$$A^{(g)} \in \mathbb{R}^{M_g \times K_g}, \qquad B^{(g)} \in \mathbb{R}^{K_g \times N_g}.$$

The total work is:

$$\text{FLOPs}_{\text{grouped}} = \sum_{g=0}^{G-1} 2M_g N_g K_g.$$

The main complication is mapping CTAs to groups and tiles. If each group g has:

$$T_g = \left\lceil \frac{M_g}{B_M} \right\rceil \left\lceil \frac{N_g}{B_N} \right\rceil$$

output tiles, then a flattened tile index p must be mapped to a group g such that:

$$\sum_{h=0}^{g-1} T_h \leq p < \sum_{h=0}^{g} T_h.$$

After finding g, the local tile index is:

$$p_g = p - \sum_{h=0}^{g-1} T_h.$$

Then:

$$t_m = \left\lfloor \frac{p_g}{T_{N,g}} \right\rfloor, \qquad t_n = p_g \bmod T_{N,g}.$$

Grouped GEMM also requires dynamic descriptor updates because the base pointers, dimensions, and strides differ by group.

Code Walkthrough

Minimal CuTe DSL Kernel Structure

The lecture describes two major decorators. A representative skeleton is:

```
import cutlass.cute as cute

@cute.jit
def launch(A, B, C, problem_shape, config):
    grid = compute_grid(problem_shape, config)
    block = compute_block(config)

    # Host-side descriptor setup would happen here.
    # Examples include TMA atoms, tiled MMA atoms, layouts,
    # shared-memory swizzles, and static kernel parameters.

    gemm_kernel(A, B, C, problem_shape, config).launch(
        grid=grid,
        block=block
    )
```

```python
@cute.kernel
def gemm_kernel(A, B, C, problem_shape, config):
    # Device-side program:
    # 1. Compute CTA coordinates.
    # 2. Slice global tensors.
    # 3. Load tiles.
    # 4. Compute.
    # 5. Store results.
    pass
```

The host-side function prepares the launch configuration and descriptors. The kernel performs device-side tiling, partitioning, memory movement, and computation.

Simple GEMV-Like Tiling

A representative conceptual kernel is:

```python
@cute.kernel
def simple_gemv_kernel(A, x, y, M: cute.Int32, K: cute.Int32):
    tid = cute.arch.thread_idx()[0]
    bid = cute.arch.block_idx()[0]

    row = bid
    acc = cute.full((), 0.0, cute.Float32)

    for k in cute.range(K):
        a_val = A[row, k]
        x_val = x[k]
        acc += a_val.to(cute.Float32) * x_val.to(cute.Float32)

    if tid == 0:
        y[row] = acc
```

This simplified code is not an optimized CuTe DSL implementation, but it captures the first example's conceptual flow: global loads, FP32 computation, and global stores.

Hardware interpretation:

- Each block computes one output row.
- Data is read from global memory.
- Computation uses scalar FMA-like operations.
- This is easy to understand but may not saturate memory bandwidth or compute throughput.

Tensor SSA Elementwise Pattern

The tensor SSA idea can be represented as:

```
# Conceptual tensor SSA flow.
fragment = cute.load(global_tensor_slice)
fragment_fp32 = fragment.to(cute.Float32)
result = fragment_fp32 * fragment_fp32
cute.store(result, output_tensor_slice)
```

The important point is that the programmer expresses operations over a tensor fragment. The compiler lowers this to vectorized operations over the local representation.

TMA Setup Pattern

A conceptual host-side TMA setup looks like:

```
@cute.jit
def prepare_and_launch(A, B, C, problem_shape, config):
    tma_atom_A = make_tma_atom_for_A(config)
    tma_atom_B = make_tma_atom_for_B(config)

    smem_layout_A = make_smem_layout_A(config)
    smem_layout_B = make_smem_layout_B(config)

    tma_desc_A = make_tma_descriptor(
        atom=tma_atom_A,
        global_tensor=A,
        smem_layout=smem_layout_A,
        tile_shape=config.tile_shape_A
    )

    tma_desc_B = make_tma_descriptor(
        atom=tma_atom_B,
        global_tensor=B,
        smem_layout=smem_layout_B,
        tile_shape=config.tile_shape_B
    )

    kernel(A, B, C, tma_desc_A, tma_desc_B, problem_shape, config).launch(
        grid=compute_grid(problem_shape, config),
        block=compute_block(config)
    )
```

Major ideas:

- `tma_atom` selects the hardware copy behavior.

- `smem_layout` determines how the tile lands in shared memory.
- The descriptor packages global shape, strides, tile shape, and destination layout.
- The kernel later uses this descriptor to issue TMA copy operations.

Device-Side TMA Partition and Copy

A representative device-side pattern is:

```
@cute.kernel
def kernel(A, B, C, tma_desc_A, tma_desc_B, problem_shape, config):
    cta_m = cute.arch.block_idx()[0]
    cta_n = cute.arch.block_idx()[1]

    # Local tile of the global tensor.
    gA_tile = local_tile(A, config.tile_shape_A, (cta_m, None))
    gB_tile = local_tile(B, config.tile_shape_B, (None, cta_n))

    # Shared-memory buffers.
    sA = allocate_shared_tensor(config.smem_layout_A)
    sB = allocate_shared_tensor(config.smem_layout_B)

    # Partition source and destination according to TMA atom.
    tAgA, tAsA = tma_partition(
        tma_desc_A, gA_tile, sA
    )
    tBgB, tBsB = tma_partition(
        tma_desc_B, gB_tile, sB
    )

    for k_tile in cute.range(config.num_k_tiles):
        cute.copy(tma_desc_A, tAgA[k_tile], tAsA[k_tile])
        cute.copy(tma_desc_B, tBgB[k_tile], tBsB[k_tile])
```

This captures the repeated pattern:

$$\text{partition} \rightarrow \text{copy}.$$

The actual CuTe DSL API names may differ depending on version, but the design pattern remains: the atom and descriptor drive the legal partitioning and copy behavior.

S2T Copy: Shared Memory to Tensor Memory

For Blackwell tensor-core workflows, some operands such as scale factors must be moved from shared memory to tensor memory.

A representative pattern is:

```
# Pick an S2T copy atom.
s2t_atom = make_s2t_copy_atom(config)

# Build tiled copy object.
s2t_copy = make_tiled_copy(s2t_atom, config.s2t_layout)

# Partition source and destination.
src = s2t_copy.partition_S(shared_scale_tensor)
dst = s2t_copy.partition_D(tensor_memory_scale_tensor)

# Copy from shared memory to tensor memory.
cute.copy(s2t_copy, src, dst)
```

The hardware interpretation is:

$$\text{Shared Scale Tensor} \rightarrow \text{Tensor-Memory Scale Tensor}.$$

This movement exists because the tensor-core instruction expects operands or metadata in a specific memory space and layout.

Tensor-Core MMA Pattern

A representative MMA pattern is:

```
mma_atom = make_mma_atom(config)
tiled_mma = make_tiled_mma(mma_atom, config.mma_layout)

a_desc = make_fragment_A(shared_A, tiled_mma)
b_desc = make_fragment_B(shared_B, tiled_mma)
accum = make_accumulator_fragment(tiled_mma)

for k_tile in cute.range(num_k_tiles):
    cute.gemm(tiled_mma, accum, a_desc[k_tile], b_desc[k_tile], accum)
```

The conceptual operation is:

$$D \leftarrow A_{\text{frag}} B_{\text{frag}} + D.$$

Major line-by-line interpretation:

- `make_mma_atom` selects a hardware tensor-core instruction shape.
- `make_tiled_mma` maps that instruction across a CTA-level tile.
- Fragment construction converts memory tensors into operand layouts expected by the MMA instruction.

- The loop over K-tiles accumulates partial products.

Tensor Memory to Register Epilogue

After tensor-core computation, the result may reside in tensor memory. The epilogue moves it to registers, optionally applies elementwise operations, and stores to global memory.

```
t2r_atom = make_t2r_copy_atom(config)
t2r_copy = make_tiled_copy(t2r_atom, config.t2r_layout)

thread_slice = t2r_copy.get_slice(thread_idx)

src = thread_slice.partition_S(tensor_memory_accumulator)
dst = thread_slice.partition_D(register_accumulator)

cute.copy(t2r_copy, src, dst)

# Optional elementwise epilogue.
reg_out = epilogue_op(register_accumulator)

cute.copy(reg_out, global_output_tile)
```

The lecture notes that a simple direct register-to-global store may work for demonstration, but a highly optimized epilogue may tile the output further and overlap tensor-memory-to-register copy with global stores.

Dual GEMM Epilogue Lambda

The dual GEMM example highlights compile-time lambda-like epilogue operations. A conceptual representation is:

```
@cute.jit
def silu(x):
    return x / (1.0 + cute.exp(-x))

@cute.jit
def epilogue(d0, d1):
    return silu(d0) * d1

@cute.kernel
def dual_gemm_kernel(A, B0, B1, C, config):
    d0 = compute_gemm(A, B0, config)
    d1 = compute_gemm(A, B1, config)

    r0 = load_accumulator_to_registers(d0)
    r1 = load_accumulator_to_registers(d1)
```

```
    out = epilogue(r0, r1)
    store_to_global(C, out)
```

The hardware-relevant concern is where $d0$ and $d1$ live. Keeping too much in registers causes register pressure; using shared memory can reduce register pressure but may compete with mainloop staging buffers.

Grouped GEMM Descriptor Update

Grouped GEMM requires dynamic TMA descriptor updates. A conceptual structure is:

```
@cute.kernel
def grouped_gemm_kernel(group_metadata, tensor_map_A, tensor_map_B,
                        tensor_map_C, config):
    tile_id = cute.arch.block_idx()[0]

    group_id, local_tile = find_group_and_tile(tile_id, group_metadata)

    A_g = get_group_pointer_A(group_metadata, group_id)
    B_g = get_group_pointer_B(group_metadata, group_id)
    C_g = get_group_pointer_C(group_metadata, group_id)

    M_g = get_group_M(group_metadata, group_id)
    N_g = get_group_N(group_metadata, group_id)
    K_g = get_group_K(group_metadata, group_id)

    # Construct or update TMA descriptors for this group.
    desc_A = update_tma_descriptor(
        tensor_map_A,
        base=A_g,
        shape=(M_g, K_g),
        strides=get_strides_A(group_metadata, group_id)
    )

    desc_B = update_tma_descriptor(
        tensor_map_B,
        base=B_g,
        shape=(K_g, N_g),
        strides=get_strides_B(group_metadata, group_id)
    )

    ensure_descriptor_update_visible()

    # Then execute a normal tiled GEMM for this group's tile.
    run_tiled_gemm_for_group(desc_A, desc_B, C_g, local_tile, config)
```

The key difficulty is that descriptors are no longer static. Each CTA must identify which group it belongs to and update the relevant tensor map information before issuing TMA loads.

Debugging Print Facilities

The lecture mentions several debugging mechanisms:

- Python `print`: useful for static compile-time information.
- `cute.print`: useful for runtime values on host or device.
- `cute.print_tensor`: useful for inspecting tensor contents, with limitations for complex layouts and narrow precision types.

A representative debugging pattern is:

```
print("Static tile shape:", config.tile_shape)

cute.print("Runtime block index:", cute.arch.block_idx()[0])

cute.print_tensor(global_tile_slice)
```

For low-precision values, the lecture suggests converting through tensor SSA into FP32 before printing:

```
frag = cute.load(low_precision_tensor_slice)
frag_fp32 = frag.to(cute.Float32)
cute.print_tensor(frag_fp32)
```

Compilation Cache Keys

CuTe DSL compilation can be expensive. The lecture describes custom cache keys so that kernels are not recompiled unnecessarily.

A conceptual pattern is:

```
def compile_key(problem_shape, static_config):
    return (
        problem_shape.num_groups,
        static_config.tile_m,
        static_config.tile_n,
        static_config.tile_k,
        static_config.num_stages
    )

@cute.jit
def compile_or_get_kernel(problem_shape, static_config):
    key = compile_key(problem_shape, static_config)
```

```
    if key in kernel_cache:
        return kernel_cache[key]

    fake_A = make_pointer(dtype=static_config.dtype, address=None)
    fake_B = make_pointer(dtype=static_config.dtype, address=None)
    fake_C = make_pointer(dtype=static_config.output_dtype, address=None)

    compiled = compile_kernel(fake_A, fake_B, fake_C,
                              problem_shape, static_config)
    kernel_cache[key] = compiled
    return compiled
```

The purpose is to separate compile-time signature construction from runtime data.

Performance Intuition

Why the Simple Example Is Not Enough

The simple GEMV-like example uses global loads, registers, FMA, and global stores. It is good for learning layout algebra, slicing, and tensor SSA, but it cannot reach peak performance for large matrix operations because it does not exploit:

- tensor cores,
- TMA,
- shared-memory reuse,
- swizzled layouts,
- software pipelining,
- warp specialization,
- multi-stage buffering.

The Main Performance Recipe

For high-performance CuTe DSL kernels, the recipe is:

Choose atoms → Choose tile shapes → Stage data through shared memory → Use tensor cores → Pipeline me

Atom Selection

The atom determines the hardware instruction shape. A poor atom wastes work or causes inefficient memory access. For example, if an MMA atom computes a

tile much wider than the actual N-dimension, many tensor-core lanes do useless work.

Selection should consider:

- problem shape,
- data type,
- alignment,
- whether M, N, or K is small,
- required scale-factor layout,
- tensor-core instruction availability.

Tile Size Trade-Off

Large tiles improve reuse:

$$I_{\text{tile}} = \frac{2B_M B_N}{b(B_M + B_N)}.$$

However, large tiles also increase:

- shared-memory usage,
- register usage,
- epilogue pressure,
- scheduling complexity,
- possible wasted work on boundary tiles.

Thus, tile size must be tuned empirically.

Software Pipelining and Warp Specialization

If a kernel does:

$$\text{load} \to \text{compute} \to \text{load} \to \text{compute},$$

then memory latency and compute latency are serialized. The goal is instead:

Time	0	1	2	3	4
Copy	K_0	K_1	K_2	K_3	K_4
Compute		K_0	K_1	K_2	K_3

This allows the steady state to approach:

$$T_{\text{per tile}} \approx \max(T_{\text{copy}}, T_{\text{compute}}).$$

Epilogue Optimization

The epilogue can become a bottleneck. If the mainloop is highly optimized but storing C is inefficient, total performance suffers. Epilogue optimization includes:

- processing smaller output subtiles to reduce register pressure,
- using vectorized stores,
- avoiding unnecessary shared-memory traffic,
- overlapping tensor-memory-to-register copy with global store,
- fusing elementwise operations such as gating or activation functions.

Small and Unconventional Matrices

For very small or unconventional matrix sizes, tensor-core utilization may be poor. In such cases, the lecture suggests focusing more on memory movement efficiency:

- choose tile shapes that minimize wasted work,
- avoid overusing tensor-core shapes that do too much empty computation,
- use pipelining even for memory movement,
- inspect memory traffic in profiling tools,
- consider whether the kernel is fundamentally bandwidth-bound.

Cluster Optimization

The lecture briefly mentions cluster-level optimization. On newer architectures, clusters can allow CTAs to cooperate and reduce global-to-local memory pressure through multicast or broadcast-like behavior.

If the same global tile is needed by multiple CTAs, cluster-level data movement can reduce redundant global memory traffic:

$$\text{Bytes}_{\text{without cluster}} = R \cdot B_{\text{tile}},$$

where R CTAs redundantly load the same tile. With cluster-level reuse:

$$\text{Bytes}_{\text{with cluster}} \approx B_{\text{tile}} + \text{local redistribution overhead}.$$

This can improve memory-bound kernels.

Nsight Compute Profiling Strategy

The lecture recommends starting with the **Speed of Light** section in Nsight Compute. The diagnostic flow is:

1. Determine whether the kernel is compute-bound or memory-bound.
2. If memory-bound, inspect global, shared, and local memory traffic.
3. Check whether there are redundant loads or excessive local-memory movement.
4. If compute-bound, inspect instruction statistics and tensor-core utilization.
5. Use source-to-SASS correlation to identify which source lines generate problematic instructions.

A useful mental checklist is:

$$\begin{aligned}
\text{Low tensor-core utilization} &\Rightarrow \text{bad MMA shape or poor feeding} \\
\text{High shared-memory bank conflicts} &\Rightarrow \text{bad swizzle or access pattern} \\
\text{High global traffic} &\Rightarrow \text{poor reuse or missing clustering} \\
\text{Register spilling} &\Rightarrow \text{epilogue or tile too large} \\
\text{Long scoreboard stalls} &\Rightarrow \text{insufficient latency hiding}
\end{aligned}$$

Common Misconceptions

Misconception: CuTe DSL Is Just Python GEMM

CuTe DSL is not simply a high-level matrix multiplication interface. It exposes low-level hardware controls. The user still selects atoms, layouts, memory spaces, and synchronization strategies.

Misconception: The Tensor Object Is Just a Pointer

A CuTe tensor is better understood as:

$$\text{Tensor} = (\text{Iterator}, \text{Layout}).$$

The iterator identifies the underlying memory access mechanism, and the layout defines coordinate-to-address mapping.

Misconception: Shared Memory Should Mirror Global Memory

For tensor-core kernels, shared-memory layout often should not mirror global-memory layout. Shared memory should be arranged for the consumer instruction's access pattern. Swizzling exists to reduce bank conflicts and satisfy alignment requirements.

Misconception: More Shared Memory Is Always Better

More shared-memory staging can improve latency hiding, but it may reduce occupancy. The trade-off is:

$$\text{More stages} \Rightarrow \text{better overlap} \quad \text{but} \quad \text{lower occupancy.}$$

Misconception: Larger Tiles Always Improve Performance

Larger tiles improve arithmetic intensity but increase shared-memory usage, register pressure, boundary waste, and epilogue cost. The best tile is problem-dependent.

Misconception: Tensor Cores Automatically Give Peak Performance

Tensor cores only provide peak throughput if they are fed efficiently. TMA, shared-memory layout, scale-factor movement, pipelining, and epilogue design all matter.

Misconception: Debugging Can Rely Only on Final Output

For layout-heavy kernels, final numerical mismatch may not reveal where the problem occurred. Intermediate inspection with `cute.print`, tensor printing, IR dumping, SASS inspection, and CUDA debugging can be necessary.

Misconception: Register Spilling Is Only a Compiler Problem

Register spilling often reflects algorithmic choices: too large an accumulator tile, too much epilogue fusion at once, or failure to subtile the epilogue.

Practical Takeaways

1. Learn CuTe layout algebra first. Most CuTe DSL code becomes understandable once tiling, slicing, and partitioning are clear.
2. Treat atoms as hardware instruction choices. Copy atoms and MMA atoms define what the hardware will actually do.
3. Use TMA for high-throughput multidimensional global-to-shared memory movement.
4. Use shared-memory swizzling to match tensor-core access patterns and avoid bank conflicts.

5. For NVFP4-style kernels, remember that scaling factors may require movement into tensor memory.
6. Optimize the mainloop through software pipelining or warp specialization.
7. Do not ignore the epilogue. It can dominate runtime if accumulator movement and stores are inefficient.
8. For grouped GEMM, descriptor update logic is the hard part. Once descriptors are correct, the computation resembles ordinary tiled GEMM.
9. Use compile cache keys to avoid recompiling for unchanged static configurations.
10. Profile with Nsight Compute starting from Speed of Light, then drill into memory, tensor-core utilization, instruction statistics, and source-to-SASS mapping.

Project or Experiment Ideas

Experiment 1: Simple GEMV to Tiled GEMV

Implement a naive GEMV-style kernel using global loads and FP32 accumulation. Then add tiling and compare:

$$T_{\text{naive}} \quad \text{versus} \quad T_{\text{tiled}}.$$

Measure bandwidth:

$$\text{Bandwidth} = \frac{\text{Bytes Read} + \text{Bytes Written}}{T_{\text{kernel}}}.$$

Experiment 2: Shared-Memory Swizzle Study

Implement two variants of a shared-memory tile load:

- unswizzled shared-memory layout,
- swizzled shared-memory layout.

Use Nsight Compute to compare shared-memory bank conflicts and tensor-core utilization.

Experiment 3: TMA Versus Manual Loads

Compare a tiled global-to-shared memory load implemented using ordinary per-thread vector loads against a TMA-based load. Measure:

$$\text{Effective Bandwidth} = \frac{\text{Tile Bytes Loaded}}{\text{Load Time}}.$$

Experiment 4: MMA Atom Shape Sweep

For several problem shapes, sweep MMA atom shapes and tile sizes. Track:

- runtime,
- TFLOP/s,
- tensor-core utilization,
- register usage,
- shared-memory usage.

Pay special attention to skinny N-dimension cases.

Experiment 5: Software Pipeline Depth

Implement a multi-stage mainloop and test stage counts $S \in \{1, 2, 3, 4\}$. Compare:

$$T(S)$$

and identify when additional stages stop helping due to shared-memory pressure or occupancy loss.

Experiment 6: Dual GEMM Epilogue Subtiling

Implement a dual GEMM with a fused epilogue:

$$C = \mathrm{SiLU}(AB_0) \odot (AB_1).$$

Compare:

- full-tile epilogue register load,
- subtiled epilogue,
- shared-memory-assisted epilogue.

Measure register spilling and runtime.

Experiment 7: Grouped GEMM Descriptor Debugging

Create a grouped GEMM with several different shapes. Validate descriptor update logic by printing tile coordinates, group indices, and selected tensor slices. Then profile the final version.

Experiment 8: CuTe DSL Versus Triton Versus CUDA

Implement the same small GEMM-like workload in:

- PyTorch eager,
- Triton,
- CUDA C++,
- CuTe DSL.

Compare implementation complexity, compile time, peak performance, and debuggability.

Final Mental Model

CuTe DSL should be understood as a Python-facing gateway into CUTLASS-style low-level GPU kernel construction. The programmer is not simply calling a library GEMM. The programmer is assembling a kernel from explicit tensor layouts, tiled partitions, copy atoms, MMA atoms, TMA descriptors, shared-memory swizzles, tensor-memory moves, pipeline stages, and epilogue logic.

The most important performance mental model is:

$$\boxed{\text{Performance} = \text{Correct Layout} + \text{Efficient Movement} + \text{Right Atom} + \text{Pipelined Compute} + \text{Careful Epilogue}}$$

The most important correctness mental model is:

$$\boxed{\text{Every tensor slice must have the layout and memory space expected by the next hardware instruction.}}$$

If the data is in the wrong layout, the tensor core may be inefficient or incorrect. If the data is in the wrong memory space, the instruction may not be legal. If the pipeline ordering is wrong, consumers may read incomplete data. If the tile shape is poorly chosen, the kernel may waste tensor-core work or overload registers and shared memory.

A strong CuTe DSL engineer thinks in four simultaneous coordinate systems:

Problem coordinates	(m, n, k, g)
Tile coordinates	(t_m, t_n, t_k)
Instruction coordinates	MMA atom fragments and copy atom lanes
Memory coordinates	global, shared, tensor memory, registers

The lecture's final lesson is that speed-of-light performance is possible only when these coordinate systems are aligned. CuTe DSL gives the programmer tools to express that alignment directly from Python, while still retaining explicit control over the hardware details that matter.

Visual Concept

Draw a pipeline diagram with global memory on the left feeding TMA into swizzled shared-memory buffers. From shared memory, show one path into tensor memory for scaling factors and another path into tensor-core MMA for matrix operands. Then show accumulators moving from tensor memory to registers, through an epilogue operation, and finally back to global memory.